algorithms

MDPI

Article

Fingerprinting Indoor Positioning Based on Improved Sequential Deep Learning

Dongfang Mao [1,2,3], Haojie Lin [4,*] and Xuyang Lou [4]

[1] College of Automation & College of Artificial Intelligence, Nanjing University of Posts and Telecommunications, Nanjing 210023, China; mdf@realidtek.com
[2] Jiangsu Vocational College of Information Technology, Wuxi 214153, China
[3] Wuxi Realid Technology Co., Ltd., Wuxi 214135, China
[4] School of IoT Engineering, Jiangnan University, Wuxi 214122, China; louxy@jiangnan.edu.cn
* Correspondence: linhj@stu.jiangnan.edu.cn

Abstract: Accurate indoor positioning is essential for many applications. However, current methods often fall short in complex environments due to signal fluctuations. We propose a new indoor positioning approach, that is, improved sequential deep learning (ISDL), to address this issue. First, we apply sequential classification algorithms to progressively narrow the search space, reducing potential location regions into smaller neighborhoods. Next, we combine a deep neural network (DNN) with Weighted K-Nearest Neighbors (WKNN) to refine the final location prediction. Then, we validate our method using the publicly available *UJIndoorLoc* dataset, demonstrating superior accuracy compared to existing methods. Specifically, we achieved 95% floor prediction accuracy and reduced the average positioning error to just 7.82 m. By combining sequential classification and the DNN-WKNN hybrid model, we achieve better localization in complex indoor environments. This system offers practical improvements for real-time location-based services and other applications requiring precise indoor positioning.

Keywords: indoor positioning; sequential deep learning; deep neural network; weighted K-nearest neighbor

check for updates

Academic Editor: Jaehyun Yoo

Received: 16 December 2024
Revised: 31 December 2024
Accepted: 1 January 2025
Published: 3 January 2025

Citation: Mao, D.; Lin, H.; Lou, X. Fingerprinting Indoor Positioning Based on Improved Sequential Deep Learning. *Algorithms* **2025**, *18*, 17. https://doi.org/10.3390/a18010017

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The rapid advancement of wireless communication and mobile device technologies has enabled numerous applications, particularly Location-Based Services (LBS) [1]. While Global Navigation Satellite Systems (GNSS) and cellular networks provide accurate positioning in outdoor environments, they struggle in indoor or obstructed areas due to Non Line of Sight (NLOS) errors [2]. Consequently, there is a growing demand for Indoor Positioning Systems (IPS) that can meet varying requirements for accuracy and coverage depending on the specific indoor environment.

To address these challenges and provide reliable indoor positioning solutions, researchers have extensively explored various technologies, as illustrated in Table 1. Among these, Radio Frequency (RF)-based positioning technologies have become particularly popular due to their unique advantages and adaptability. Within RF technologies, two main approaches are widely used: ranging-based techniques and proximity detection techniques. Ranging-based techniques [3] determine the precise distance between devices by measuring signal characteristics. Common methods in this category include Angle of Arrival (AoA) [4], Time of Arrival (ToA) [5], and Received Signal Strength Indicator

(RSSI) [6]. These technologies hold a significant position in the field of indoor positioning because of their high precision and wide range of application scenarios.

Among ranging-based techniques, AoA and ToA offer superior accuracy for indoor positioning systems. AoA can achieve centimeter-level precision and supports three-dimensional positioning, making it highly suitable for applications requiring fine-grained localization. ToA provides sub-meter accuracy with improved reliability and reduces the need for site-specific calibration, enhancing its practicality in various environments. However, both methods have practical limitations. Despite that some AoA methods like the I/Q method for full-hardware localization [7] may reduce costs and deployment workload, AoA still necessitates precise control of antenna arrays, introducing significant implementation challenges. Additionally, AoA algorithms usually rely on in-phase and quadrature data, leading to substantial resource consumption. ToA requires more expensive hardware, precise calibration, and ongoing maintenance. These factors make both methods less feasible for large-scale or cost-sensitive indoor positioning systems.

Table 1. Overview of indoor positioning technologies.

Technology	Method	Advantage	Disadvantage
Statellite-based [1]	GPS	Easy to navigate Low cost implementation	Cannot be implemented in an indoor environment
Magnetic-based [8]	(1) Geomagnetic Field (GMF) (2) Artificially Magnetic Field (AMF)	High accuracy	High cost implementation Complex
Optical-based [9]	(1) Infrared (2) Visual light communications	Low cost implement	Complex Low accuracy
Vision-based [10]	Camera system	Low cost implement	Complex Low accuracy
Radio frequency [11–13]	(1) Wireless Fidelity (Wi-Fi) (2) Bluetooth Low Energy (BLE) (3) Radiofrequency Identification (RFID) (4) UltraWideBand (UWB)	Low cost implement Low power consumption Small size High accuracy	Limited coverage Strong environmental dependence Complex

In contrast, fingerprint-based indoor positioning, particularly Wi-Fi-based approaches, has emerged as a more accessible and cost-effective alternative. This method utilizes received Wi-Fi signals, such as the RSSI, from commonly available Access Points (APs) as unique location fingerprints. Unlike Global Positioning Systems (GPS), which struggles to penetrate indoor environments, Wi-Fi fingerprinting takes advantage of existing infrastructure without requiring additional hardware. Location fingerprints, including RSSI, Channel State Information (CSI), or geomagnetic field intensity, provide distinctive signatures for each location. The process typically consists of two stages: during the offline stage, RSSI measurements are collected at predefined Reference Points (RPs) and stored in a database. In the online stage, the user's position is estimated by comparing newly acquired RSSI values to the pre-recorded data [14,15]. This approach, leveraging readily available Wi-Fi infrastructure, strikes a balance between accuracy, cost, and scalability, making it ideal for indoor positioning applications.

One benefit of Wi-Fi RSSI fingerprinting is its affordability, as it leverages existing Wi-Fi networks without the need for costly sensors or additional infrastructure. CSI, while offering fine-grained localization, requires specialized hardware, making RSSI the more practical choice for widespread applications. However, RSSI is susceptible to multi-path effects and signal fading [16], impacting localization accuracy [17]. To address this, methods like the Kalman filter [18,19] are often combined with Gaussian-based approaches to filter out interference and provide a stable RSSI value for accurate distance estimation. A Fourier-transform Fuzzy C-means Kalman filter [20] has also been proposed to further improve the stability of RSSI fingerprints for indoor positioning.

The incorporation of Artificial Intelligence (AI) and Machine Learning (ML) methods has significantly improved indoor positioning systems. [21]. While algorithms like K-Nearest Neighbors (KNN) are widely used for their simplicity, their performance declines in complex, large-scale, multi-floor environments [22]. In contrast, Deep Neural Networks (DNN) can model the complex relationships between RSSI and location, improving accuracy in large-scale settings [23]. Recent advancements have focused on DNNs that learn RSSI distributions rather than directly converting RSSI values into coordinates, with further exploration of Convolutional and Recurrent Neural Networks (CNN and RNN) [24,25] to improve robustness. A complex neural network [26] based joint estimation of Angle of Arrival (AoA) and Angle of Departure (AoD) has already leveraged advanced deep learning architectures to explore the positioning capabilities within the communication infrastructure of 6G systems. A Wi-Fi Generative Adversarial Networks (GANs) [27,28] has also been introduced, generating labeled fingerprint data and achieving higher accuracy than standard supervised DNN.

Small-scale indoor localization is relatively straightforward, limited to single floors or enclosed spaces with a few access points. However, scaling to multi-building and multi-floor environments introduces significant complexity. Binary classification techniques, as introduced in [29], use features derived from real-world coordinates to partition the location space, applying tree-based classifiers sequentially to progressively narrow down the search area.

In this paper, we propose a novel fingerprint-based area localization scheme that applies sequential deep learning to iteratively reduce the search space. By integrating a deep neural network with Weighted K-Nearest Neighbors (WKNN), We enhance location estimates by utilizing the statistical patterns of RSSI samples at each RP. Our method is evaluated using the publicly available UJIIndoorLoc dataset.

The rest of this paper is organized as follows: Section 2 covers the fundamentals of the fingerprint-based positioning method, along with an overview of the conventional WKNN algorithm and DNN-based positioning model. Section 3 provides a detailed explanation of the implementation process for our proposed localization algorithm. Section 4 we present the localization results obtained using our method on the $UJIIndoorLoc$ validation dataset. Finally, Section 5 concludes this work.

2. Preliminaries

In this section, we present the fundamental concept of fingerprint-based RSSI positioning. We then describe the conventional positioning algorithms, including the WKNN method and DNN-based regression (DNNR).

2.1. Fingerprint-Based Positioning

The first step in fingerprint-based positioning is the construction of a reference map, where a specific RP serves as the origin of the environment. At each predetermined coordinate, known as an RP, the RSSI values from various APs are recorded. Notably, the exact locations of the APs do not need to be known, as we assume the APs remain stationary, and their RSSI values serve as unique fingerprints for each RP. These RSSI measurements, taken multiple times at each fixed RP, form the input for positioning algorithms, with the corresponding RP locations as the output. The raw RSSI dataset for all RPs is defined as [29]

$$S_i^j = m(L_i, \epsilon_i^j, \eta), \tag{1}$$

where $S_i^j \in \mathbf{R}^R, i = 1, \cdots, M, j = 1, \cdots, N_j$ represents the R-dimensional RSSI measurements, $L_i \equiv (\ell_{oi}, \ell_{ai}, \ell_{fi}, \ell_{bi})$ denotes the location information of the mobile device, which includes Longitude, Latitude, Floor, and Building for the ith observation. Additionally, $\epsilon \in \mathbf{R}^{d_\epsilon}$ captures the unobserved features, while $\eta \in \mathbf{R}^R$ represents the error term in the model. The vector-valued function m characterizes the structural connection between the mobile device's observed and unobserved features and the associated R dimensional RSSI measurements.

Two types of noise are present in the raw RSSI dataset shown in ϵ: small-scale fading and large-scale fading. Large-scale fading, which is position-dependent and nearly time-invariant, cannot be mitigated by simply averaging samples over time [16]. As a result, it persists in the dataset. Conversely, small-scale fading can be reduced by averaging the RSSI samples. The averaged RSSI values from the raw dataset in (1) can be represented as

$$\bar{S} = \begin{bmatrix} \bar{S}_1 \\ \vdots \\ \bar{S}_M \end{bmatrix}_{M \times R} \tag{2}$$

where $\bar{S}_i$ denotes the average RSSI value for all the AP in the ith RP. Many traditional positioning algorithms prefer to use this averaged dataset instead of the raw RSSI data, as it helps mitigate the effects of small-scale fading and reduces the computational time during the search process.

After the fingerprint map is established, the user's current location is determined by comparing real-time data with the stored fingerprints. This process can be enhanced using machine learning algorithms such as WKNN or more advanced techniques like DNN. DNN improves accuracy by learning complex relationships between the RSSI and the user's location during the training phase.

2.2. Conventional WKNN Algorithm

A widely used matching algorithm in fingerprint-based positioning is the WKNN algorithm [30]. WKNN selects k RPs with the smallest Euclidean distances based on RSSI measurements, and the user's estimated location is determined by calculating the weighted average of these k RPs:

$$\left[\hat{\ell}_o, \hat{\ell}_a, \hat{\ell}_f, \hat{\ell}_b\right]^\top = \sum_{k}^{i=1} w_i \left[\ell_{oi}, \ell_{ai}, \ell_{fi}, \ell_{bi}\right]^\top \tag{3}$$

where $\left[\hat{\ell}_o, \hat{\ell}_a, \hat{\ell}_f, \hat{\ell}_b\right]^\top$ is the estimated user location, and w_i is the weight assigned to the position of the ith RP, determined based on the RSSI Euclidean distance [31].

The Enhanced Weighted K-Nearest Neighbor (EWKNN) algorithm [32] enhances the traditional WKNN by adaptively determining the value of k and incorporating an RSSI Euclidean distance threshold. This threshold ensures that only reliable RPs, with sufficiently small distances, are used in the localization process, thereby enhancing the precision and reliability of the positioning system. Additionally, a Bayesian localization algorithm was introduced in [33], which leverages Bayesian classification theory to provide a probabilistic approach to localization, maximizing the information obtained from the received RSSI vector.

2.3. Conventional DNN-Based Positioning

The traditional DNNR positioning algorithm is trained to map RSSI vectors to Cartesian coordinates. It operates with R input nodes representing the RSSI samples and two output nodes corresponding to the ℓ_o and ℓ_a coordinates. As the model aims to predict locations using signals from APs, the number of input nodes corresponds to the number of APs. The DNN architecture includes an activation function f^l, weight matrix W_l, and input vector v_l for the lth layer. During training, the model optimizes the layer weights to minimize the mean squared error (MSE) between the actual and predicted coordinates, applying the following objective function [30]:

$$\mathcal{L}_{MSE} = \frac{1}{M} \sum_{M}^{i=1} \left(\left(\ell_{oi} - f^l_{\ell_o}(i)\right)^2 + \left(\ell_{ai} - f^l_{\ell_a}(i)\right)^2 \right) \tag{4}$$

where ℓ_{oi} and ℓ_{ai} are the actual coordinates of the rth RP, and $f^l_{\ell_o}(i)$ and $f^l_{\ell_a}(i)$ are the predicted coordinates for the RP. The training process involves minimizing this error through backpropagation [34], which consists of two phases: forward and backward propagation. Once optimized, the trained DNN model can directly map test RSSI samples to Cartesian coordinates.

However, this traditional approach overlooks the statistical characteristics of RSSI samples at each RP. In the next section, we present our proposed method, which integrates all RSSI samples during the training phase to improve positioning accuracy.

3. System Overview

Our proposed system consists of region estimation and coordinate estimation. In the region estimation phase, a sequence of binary classification algorithms gradually locates the relevant region. In the position estimation phase, the specific coordinates are determined by the proposed fingerprint-based deep neural network architecture. Figure 1 illustrates the overall framework of our method, providing a visual overview of both phases. Next, we present more details of each phase.

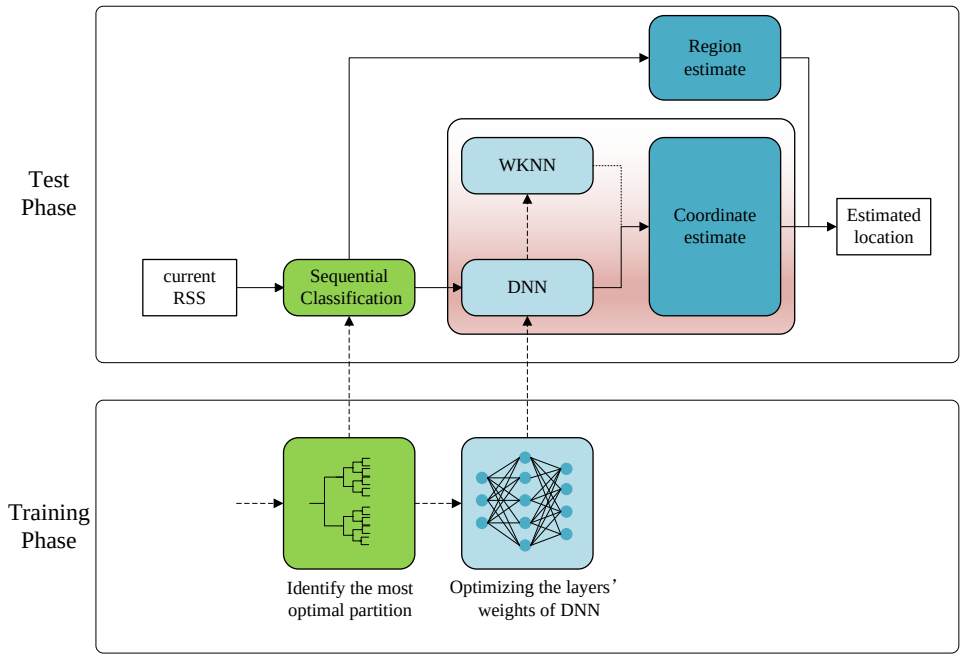


Figure 1. The whole architecture of the proposed method.

3.1. Region Estimation

Consider the following general nonparametric model for the RSSI readings: for each observation $i = 1, \cdots, N$, the model is represented as follows:

$$S_i = m(L_i, \epsilon_i, \eta), \tag{5}$$

where $S_i \in \mathbf{R}^R$ is R dimensional RSSI measurements, it is similar to (1). However, in the method proposed in this article, the values measured at the same location are treated as separate data points rather than being averaged in (2). Additionally, we use m_r to represent the rth component of m, which connects the RSSI measurement from the rth AP to its corresponding structural inputs.

$$S_{ir} = m_r(L_i, \epsilon_i, \eta_r). \tag{6}$$

Moreover, based on the marginal distribution of the structural inputs $(L_i, \epsilon_i, \eta_r)$, $\mathbb{E}(L_i|S_i)$ can be derived using Bayes' rule. We define $h_0(\cdot) \equiv \mathbb{E}(L_i|S_i = \cdot)$ as the infinite-dimensional function of interest to be estimated. Unlike traditional machine learning localization approaches, which directly estimate $h_0(\cdot)$ from large training samples and use the approximation $\hat{h}(S_i)$ to predict L_i, the sequential learning algorithm, as outlined in Steps 1–4 in Section 3.1, offers a more reliable solution.

First, partition the location space L into two regions, $\mathcal{L}_0$ and $\mathcal{L}_1$, using a simple hyperplane, such that $\mathcal{L} = \mathcal{L}_0 \cap \mathcal{L}_1$ and $\mathcal{L}_0 \cup \mathcal{L}_1 = \emptyset$. Define a dummy variable $Z_i = 1(L_i \in \mathcal{L}_1)$, where $Z_i = z$ indicates that location L_i is within $\mathcal{L}_z$. While we use a binary partition here, $\mathcal{L}$ could be divided into multiple categories $(K \geq 2)$. By utilizing the feature Z_i, we apply a binary classification algorithm to infer Z_i from S_i, which is a simpler task compared to the original localization problem. By selecting an appropriate partition, we aim for near 100% accuracy. Next, define our target function as $h_0^*(S_i, z) \equiv \mathbb{E}(L_i|S_i, Z_i = z)$ for $z = 0, 1$, and estimate $h_0^*(\cdot, z)$ using machine learning. Notably, we focus on subsamples with $Z_i = z$ during training, rather than using the entire dataset. Finally, we divide the training data $\mathcal{T}$ into N_F regions, denoted as $\mathcal{L}_{\mathcal{F}_j}$, where $j = 1, \cdots, N_F$.

Remark 1. *These regions could represent a building or a specific office on a particular floor within a building.*

This approach offers two significant advantages. First, the relationship captured by $h_0^*(\cdot, z)$ is inherently simpler and more straightforward for model selection, as it focuses on how L_i depends on S_i within the specific context of $Z_i = z$. This makes the estimation process more manageable compared to the broader $h_0(\cdot)$ relationship. Second, by training on subsamples where $Z_i = z$, the method effectively reduces noise from locations outside L_z. This localized approach is especially effective in addressing observational-level heterogeneity caused by mobile devices, resulting in more reliable outcomes.

Remark 2. *The classification algorithm is required to achieve quite perfect accuracy. If no binary partition meets this condition, the partitioning of the location space should be halted. This requirement indicates that the variable Z_i, as a binary feature of L_i, should have a deterministic relationship with S_i. By keeping the binary feature variable Z_i fixed, we can control the high-dimensional S_i within a localized area.*

Step 1. Input: Observations $\{(S_i, L_i)\}_{i \in \mathcal{T}}$ and $\{(S_i, L_i)\}_{i \in \mathcal{V}}$.
Step 2. Initialization: Start with an unlabeled root node associated with L_0.
Step 3. While there is an unlabeled leaf $\mathcal{L}_{\mathcal{F}}$, do the following:
- Step 3.1. Partition: Split $\mathcal{L}_{\mathcal{F}}$ into $\mathcal{L}_{\mathcal{F}0} \cup \mathcal{L}_{\mathcal{F}1}$ across potential partitions.
- Step 3.2. For each partition:
  - i. Assign the observations $L_i \in \mathcal{L}_{\mathcal{F}}, i \in \mathcal{T}$ to the corresponding leaf $\{\mathcal{L}_{\mathcal{F}0}, \mathcal{L}_{\mathcal{F}1}\}$.
  - ii. Label the leaf observations using $Z_i = 1(L_i \in \mathcal{L}_{\mathcal{F}1})$.
  - iii. Train a binary classification model on $\{(Z_i, S_i) : L_i \in \mathcal{L}_{\mathcal{F}}; i \in \mathcal{T}\}$.
  - iv. Calculate the accuracy τ using the validation set $\{(Z_i, S_i) : L_i \in \mathcal{L}_{\mathcal{F}}; i \in \mathcal{V}\}$.
- Step 3.3. Select partition: Choose the partition $\mathcal{L}_{\mathcal{F}0} \cup \mathcal{L}_{\mathcal{F}1}$ with the highest τ.
- Step 3.4. If stopping condition is met:
  - i. Label the leaf $\mathcal{F}$ by $\mathcal{L}_{\mathcal{F}}$.
- Step 3.5. Else:
  - i. Add $\mathcal{L}_{\mathcal{F}0}$ and $\mathcal{L}_{\mathcal{F}1}$ as new unlabeled leaves.

Step 4: End while.

3.2. Position Determination

We have partitioned the training data into N_F regions using sequential classification algorithms. For a given test fingerpoint S_0, we apply the same algorithm to determine its region number j. Then, we select all M_j RPs within that region, corresponding to the training data $\{S_i\}_{i \in \mathcal{T} \cap \mathcal{F}_j}$. Next, we will use these training data for position determination.

3.2.1. DNN

In our improved DNN architecture, the input layer remains consistent with that of a traditional DNNR model. However, the number of output nodes is aligned with the number of RPs, as illustrated in Figure 2. The architecture utilizes all RSSI samples from the RPs during the optimization process to capture the statistical characteristics of the RSSI samples at each RP. Each RP is treated as a unique class, and the RSSI sample distributions at these RPs are modeled throughout optimization. Unlike traditional approaches that rely on averaged RSSI datasets, our method leverages the complete RSSI samples, as

shown in (5). The softmax activation function is applied at the output layer, restricting the output values to a range between 0 and 1, while ensuring their sum equals 1. This enables the output values to be interpreted as probabilities corresponding to each RP. Effectively, the improved DNN acts as a multivariate probability density estimator, assigning higher weights to similar samples and extracting valuable information even from a single RSSI sample. The output for the rth node in the output layer is calculated as follows:

$$\text{SoftMax} \rightarrow f_L^r = \frac{exp(v_L^r)}{\sum_{q=1}^{R} exp(v_L^q)} \tag{7}$$

where $\sum_{q=1}^{M_j} f_L^r = 1$. Instead of using the coordinate of RPs as output labels, we use one-hot encoded labels for RPs as follows

$$C = \{\underbrace{[1,0,\cdots,0]_1^\top}_{c^1}, \underbrace{[0,1,\cdots,0]_2^\top}_{c^2}, \cdots, \underbrace{[0,0,\cdots,1]_{M_j}^\top}_{c^{M_j}}\} \tag{8}$$

where c^r is the label for samples of the rth RP. The objective function proposed during the optimization process is either the log-likelihood function or the cross-entropy function, defined as [35]:

$$\mathcal{L} = -\sum_{i=1}^{M_j N} \sum_{r=1}^{M_j} [c^r]_i log([f_L^r]_i) \tag{9}$$

where M_j is the number of the RPs in region j, N is the number of samples at each RP, $[c^r]_i$ is the desired output of the ith sample at the rth node, and $\left[f_L^r\right]$ is the predicted output for the ith sample at the rth node. The cross-entropy loss function does not penalize output nodes with true values of 0. However, it imposes a logarithmic penalty on the target output nodes when the predicted value is significantly less than 1. This means the log function favors a strong selection of a single class, which is crucial for optimizing the weights of the network layers. The goal is to maximize the probability assigned to the closest RP as much as possible.

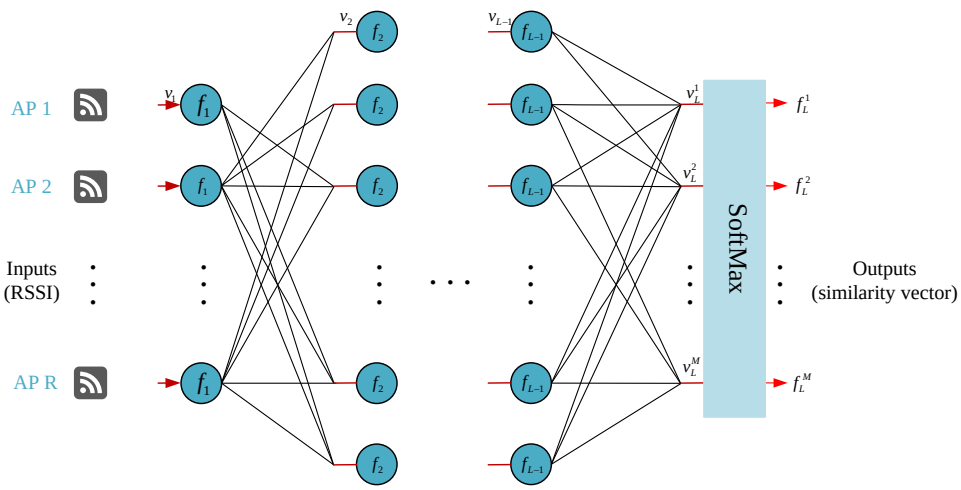


Figure 2. Proposed DNN architecture.

The computational complexity of the improved DNN can be analyzed as follows: The number of input nodes equals R, the number of layers equals L, and the number of output nodes equals the number of the RPs in region j denoted as M_j. Additionally, the number of nodes in each layer l is represented as h_l, where $h_1 = R$ (input layer) and $h_L = M_j$ (output layer). For the layers up to $L-2$, the computational complexity is equivalent to that of a conventional DNN used for regression tasks, given by $\mathcal{O}\Big(\sum_{i=1}^{L-2} h_i h_{i+1}\Big)$. At the $(L-1)$th layer, two primary factors contribute to the computational complexity: the activation function and the matrix weight multiplication. The activation function has a complexity of $a(h_{L-1})$, while the matrix weight multiplication is influenced by the number of RPs, resulting in a combined complexity at this layer of $\mathcal{O}(a(h_{L-1}) + h_{L-1}M_j)$. In the final layer, the computation of the softmax activation function introduces additional dependencies among the outputs. Assuming the complexity of the exponential function (exp) is comparable to that of a multiplication—due to its approximation via a finite number of multiplications in its Taylor series expansion—the complexity of the softmax function becomes $\mathcal{O}({M_j}^2)$. Taking into account all contributions across the layers, the total computational complexity of the improved DNN is expressed as follows:

$$\mathcal{O}\Big(\sum_{i=1}^{L-2} h_i h_{i+1} + h_{L-1}M_j + {M_j}^2\Big).$$

This overall complexity highlights that the efficiency of the improved DNN is influenced by the number of RPs.

3.2.2. WKNN

When the RP distribution is relatively uniform within a region, the DNN training performance will be excellent. Conversely, if the RP distribution is uneven or sparse, the DNN's training effectiveness will decline. To address this, during the model training phase, we set a decision threshold: if the final loss function value exceeds this threshold, the WKNN algorithm is invoked.

Step 1. Input: M_j training data $\{S_i\}_{i \in \mathcal{T} \cap \mathcal{F}_j}$, a test fingerprint S_0, and the number of neighbours k.

Step 2. Identify $\{S_0^1, S_0^2, S_0^3\}$ as the first, second and third strongest APs in S_0.

Step 3. For each S_i in $\mathcal{T}_j = \mathcal{T} \cap \mathcal{F}_j$, do the following:

- Step 3.1. Identify $\{S_i^1, S_i^2, S_i^3\}$ as the first, second and third strongest APs in S_i.
- Step 3.2. If $\{S_i^1, S_i^2, S_i^3\}$ is equal to $\{S_0^1, S_0^2, S_0^3\}$:
  - i. Add S_i to $\mathcal{T}_j'$.
- Step 3.3. If $\mathcal{T}_j'$ is empty:
  - i. Set $\mathcal{T}_j'$ equal to $\mathcal{T}_j$.

Step 4. For each S_i in $\mathcal{T}_j'$:

- Step 4.1. Let $B(i)$ be the number of APs observed in both S_i and S_0.
- Step 4.2. Calculate $C(i)$ using the formula:

$$C(i) = \frac{1}{R}\sum_{r=1}^{R} |S_{ir} - S_{0r}| - 2B(i)$$

Step 5. Select the k RPs with the smallest $C(i)$ values, denoted as $\mathcal{T}_j''$.

Step 6. For each S_i in $\mathcal{T}_j$:

Step 6.1. If $S_i \in \mathcal{T}_j''$:

i. Calculate $d_i = \text{EuclideanDistance}(S_0, S_i)$.

ii. Set $w_i = \frac{1}{(d_i)^2}$.

Step 6.2. Else:

i. Set $w_i = 0$.

Step 7. For each S_i in $\mathcal{T}_j$:

Step 7.1. Normalize the weights as follows:

$$W_i = \frac{w_i}{\sum_{i=1}^{M_j} w_i}$$

Step 8. Output: Return W.

Steps 1–8 in Section 3.2.2 illustrates the process of enabling WKNN to determine the weight $W \in \mathbb{R}^{M_j \times 1}$ when DNN is not applicable. First, the trained classification network is used to predict the region where the test point resides, and all training data within that region are extracted. Next, the sequence numbers of the three strongest APs are identified for both the test point and the training data, with the matching training data recorded. Then, similarity values are calculated for these training data, and the k most similar training points are selected. Finally, the distance function, typically the Euclidean distance, is used to calculate the RSSI distance and weight.

3.2.3. Coordinate Estimation

Upon receiving a test fingerprint S_0, the first step is to determine its corresponding region using the algorithm described in Section 3.1. Let the algorithm identify the region as $\mathcal{T}_j$, which contains M_j RPs. Subsequently, the similarity vector p can be computed by applying either the proposed DNN or the WKNN method, as follows:

$$p = \begin{cases} f_L, & \text{if} \mathcal{L} < \beta \\ W, & \text{otherwise.} \end{cases} \tag{10}$$

where $p = \left[p_1, p_2, \cdots, p_{M_j}\right]^\top$, f_L represents the output of the DNN, and W corresponds to weights obtained from WKNN. Once the similarity vector p is obtained, the estimated location coordinates $(\hat{\ell}_o, \hat{\ell}_a)$ can be computed as follows:

$$\hat{\ell}_o = \ell_o p \tag{11}$$

$$\hat{\ell}_a = \ell_a p \tag{12}$$

where $\ell_o = \left[\ell_{o1}, \ell_{o2}, \cdots, \ell_{oM_j}\right]^\top$ and $\ell_a = \left[\ell_{a1}, \ell_{a2}, \cdots, \ell_{aM_j}\right]^\top$ represent the longitude and latitude coordinates of training data $\{S_i\}_{i \in \mathcal{T}_j}$, associated with region $\mathcal{T}_j$.

4. Experimental Results and Discussion

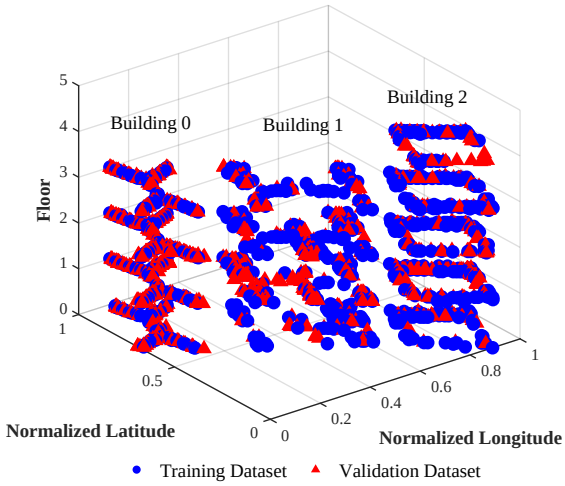
4.1. Experimental Testbed

To evaluate the localization accuracy of the proposed algorithm, we employ the $UJIndoorLoc$ dataset, which spans over 108,000 m^2 across three to five floors of buildings on the University Jaume I (UJI) campus in Spain [36]. The dataset consists of a training set with 19,937 records and a validation set with 1,111 records, encompassing nine fields that capture location and measurement information Specifically, the nine dimensions include

coordinates, floor, building, space, relative position, user ID, device ID and timestamp encapsulating details such as where, who and when the sample was collected. Notably, nine out of the 11 devices used in the validation dataset are distinct from those used in the training dataset, highlighting its robustness for assessing generalization. To ensure statistical independence between the training and validation datasets, the validation data were collected three months after the training data. The localization environment includes 520 wireless APs spread across three buildings (Buildings 0, 1, and 2), each consisting of four to five floors. Each observation in the dataset contains real-world coordinates (longitude, latitude, and floor) of a mobile device, along with 520 RSSI values from all detected APs. Figure 3 displays the floor plans of the three buildings on the UJI campus map, along with their respective buildings, as well as the Spatial distributions of the dataset for LONGITUDE, LATITUDE, and FLOOR.

The RSSI values range from -104 dBm, representing the weakest signal, to 0 dBm, indicating an exceptionally strong signal. If a mobile device does not detect an AP's signal, it is coded as -105 dBm. For simplicity, we denote N = 19,937 and $R = 520$ as training sample size and the number of APs (features), respectively.

(**a**)



(**b**)

Figure 3. Spatial distributions. (**a**) Buildings on the UJI campus map from Google Earth. (**b**) Spatial distributions of training and validation dataset.

To enhance the performance of machine learning algorithms, it is essential to clean and preprocess raw data to handle missing values, noise, and improper formats. First, non-detected RSSI values are replaced with −105 dBm, a value lower than the weakest detectable signal. Then, any AP whose RSSI remains constant across the training and/or validation datasets is removed from the analysis [37]. Additionally, we separate the training samples into two folders based on the collection time of the fingerprints. If the RSSI values from the same AP for the same RP show significant fluctuations between folders, we exclude their readings since they suggest the AP's physical location might have changed during data collection. This noise reduction and filtration process reduces the number of RSSI readings R from 520 to 300, significantly improving both the accuracy and speed of our learning algorithms.

In Table 2, we provide a summary containing the essential information regarding the experimental testbed.

Table 2. Summary of information about the experimental tested.

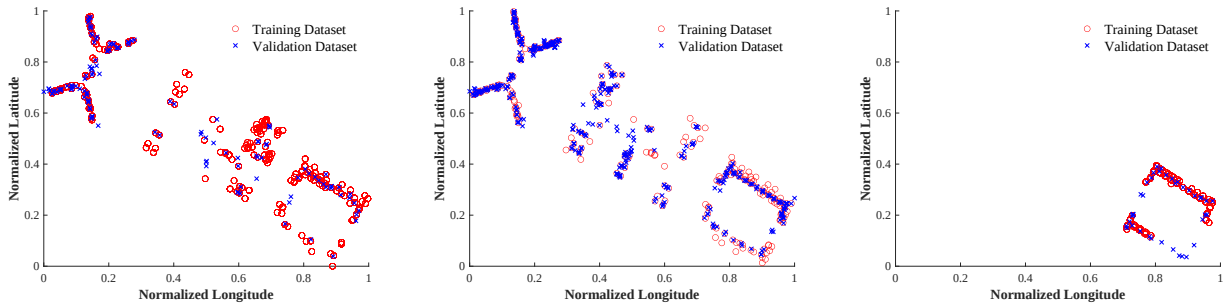
Parameter	Value
Distinct buildings	3
Distinct floors	13
Traing samples	19,937
Traing points	933
Traing device	16
Test samples	1111
Test device	13

4.2. Results Based on Experimental Data

We first apply a sequential classification algorithm to identify the building and floor of a test point. Based on the estimated location (building and floor number), the training data are filtered and normalized. The filtered data are then used to train the proposed DNN model. If the DNN model achieves accuracy above the required threshold, the corresponding weight vector for the test point is output. Otherwise, the proposed WKNN algorithm is employed to generate the weight vector, which is subsequently used to estimate the specific coordinates of the test point.

Several DNN models were trained using data collected from different regions, with the loss function values reported in Table 3. The loss function threshold was set to 0.15. For any test point where the DNN model's loss function for the estimated region exceeds this threshold, the proposed WKNN algorithm was applied instead. However, as illustrated in Figure 4, the DNN model demonstrated suboptimal performance on floors 0 and 1 of Building 1, as well as on floor 4 of Building 2. This performance deficiency can be attributed to the uneven distribution of training data across these floors and the scarcity of data in certain areas. For example, the lack of training data on the lower right part of floor 4 in Building 2 negatively impacted the positioning accuracy.

To assess the practicality of our algorithm in real-time localization applications, we analyzed the time required to obtain the first position estimation from the moment a test data point is input. After all networks were trained, our results indicated that the time-to-first estimation is approximately 0.23 s when using MATLAB 2024a for simulation on hardware with the following specifications: an Intel Core i7-13700KF CPU and an NVIDIA GeForce RTX 4060 Ti GPU. This processing time satisfies real-time positioning requirements, demonstrating the feasibility of our algorithm for real-time indoor localization scenarios. Therefore, our method can be effectively utilized in settings where timely location information is critical.



(**a**) Floor 0 (**b**) Floor 1 (**c**) Floor 4

Figure 4. Data distribution on selected floors.

Figure 5 provides a visual comparison between the actual positions of the validation dataset and the predicted positions generated by our method. To accurately assess the prediction error, discrepancies in building and floor estimates were also considered with a uniform floor height of 4 m assumed during the evaluation. Figure 6 displays the positioning errors for test points across different floors, while Table 4 summarizes the mean positioning error for each floor across different buildings.

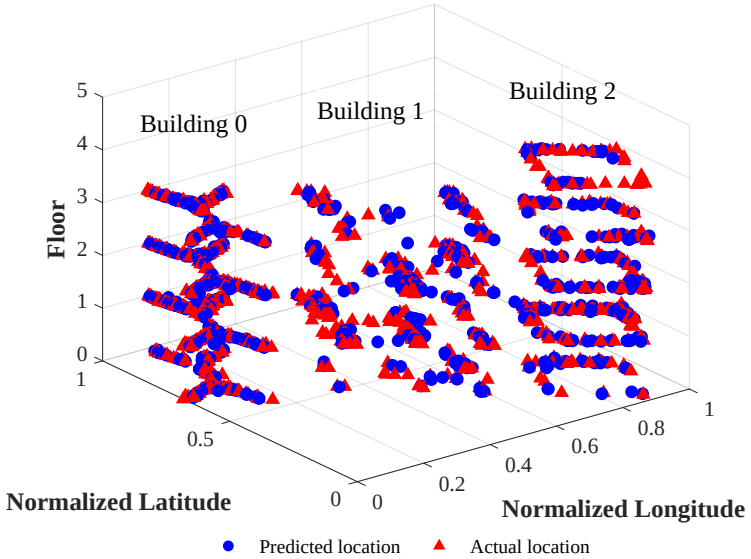
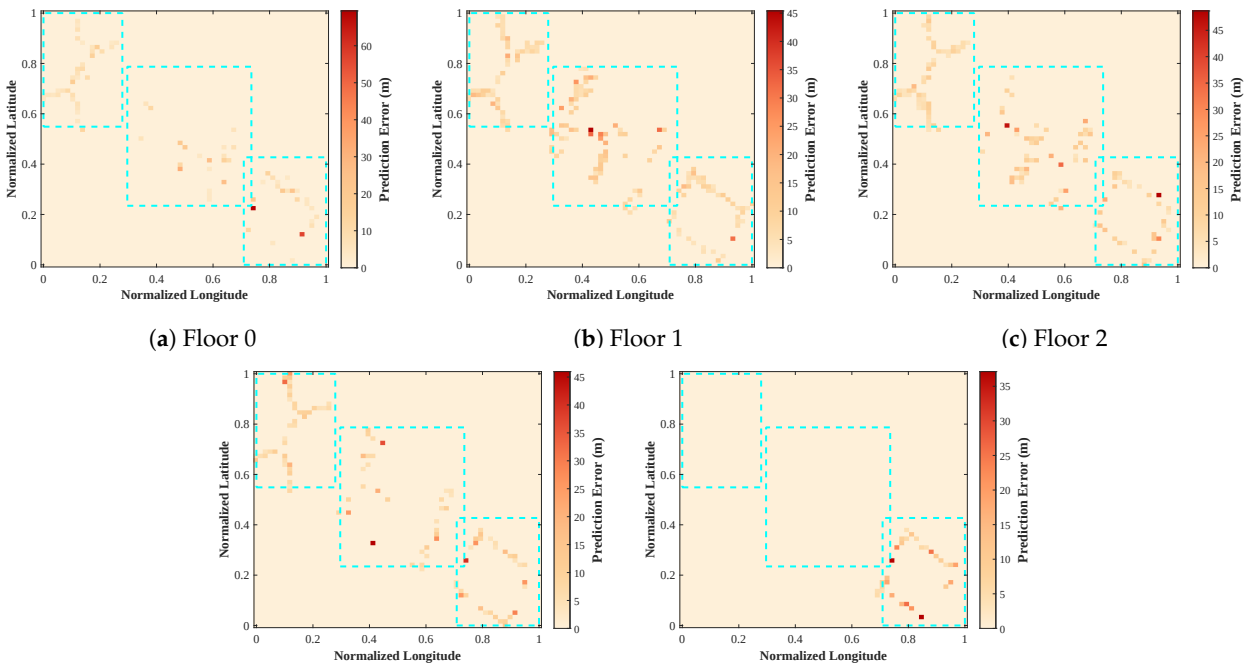


Figure 5. Actual location and predicted results of validation dataset.

From these results, we observed that the proposed method consistently performed well in Building 0 which is attributed to the reasonable and balanced distribution of the training points collected in this region. Notably, the test set was collected three months after the training set, during which environmental changes—such as the movement of large furniture and the presence of moving people—might have occurred. Despite these potential changes, our method demonstrated robust localization performance in the test, indicating a degree of resilience to typical environmental dynamics within indoor settings.

However, we acknowledge that significant layout modifications involving extensive changes, such as major furniture rearrangements or structural alterations, may lead to outdated fingerprint data and ultimately degrade positioning accuracy. This limitation could also explain the lower positioning performance observed on certain floors in our simulations. Currently, our proposed method cannot fully handle scenarios involving such extensive layout changes, and addressing this challenge remains an area for future work.



(**a**) Floor 0 (**b**) Floor 1 (**c**) Floor 2

(**d**) Floor 3 (**e**) Floor 4

Figure 6. Prediction errors across selected floors. The blue boxes represent the buildings (from left to right: Buildings 0, 1, and 2).

Table 3. The final loss function of proposed DNN model.

Building	Floor	Loos Function Value
0	0	0.0910
	1	0.0573
	2	0.0529
	3	0.0314
1	0	0.1756
	1	0.2491
	2	0.0058
	3	0.0719
2	0	0.0289
	1	0.0407
	2	0.0387
	3	0.0131
	4	1.3959

Additionally, a comparison between our localization results and those of the EvAAL 2015 competitors [37] is presented in Table 5. This comparison evaluates performance based on mean positioning error, floor hit rate, and building hit rate. Notably, the proposed algorithm achieved a 100% accuracy rate in building estimation and an impressive overall floor hit rate of 95.05%. Furthermore, the algorithm's performance falls between that of the top-ranking teams in the EvAAL 2015 competition, demonstrating its effectiveness and competitiveness in the field.

Table 4. Building-floor mean positioning error.

Building	Floor	Mean Positioning Error (m)
0	0	5.62
	1	5.81
	2	5.67
	3	7.41
1	0	9.64
	1	10.23
	2	10.39
	3	8.27
2	0	13.45
	1	6.11
	2	9.84
	3	9.88
	4	12.34

Table 5. Localization results on *UJIndoorLoc* validation dataset vs. six best methods in [36,37].

Method	Building Success Rate	Floor Success Rate	Mean Positioning Error (m)
ISDL	100%	95.05%	7.82
SCNN	100%	95.52%	9.68
RTLSUM	100%	93.74%	6.20
ICSL	100%	86.93%	7.67
HFTS	100%	96.25%	8.49
MOSAIC	98.65%	93.86%	11.64
KNN	99.73%	89.92%	7.9

5. Conclusions

In this work, we presented a novel fingerprint-based indoor positioning algorithm that enhances positioning accuracy in multi-building, multi-floor environments. Unlike traditional machine learning approaches that train models on the entire dataset, the tree-based binary classification method introduced here enables the selection of a subset of training samples for localization. It significantly reduces the complexity of the relationships that the algorithm must learn in its final stage. Conventional DNN-based positioning algorithms typically overlook the statistical characteristic of RSSI samples at each RP and rely on two output nodes to directly estimate the user's location. In contrast, the improved DNN model achieves higher accuracy by incorporating one output node per RP, allowing it to capture the variations in RSSI patterns during the optimization process. Additionally, the designed WKNN algorithm was employed to address the limitations of DNN models when training data is inadequate, further enhancing localization accuracy. Using the publicly available UJIIndoorLoc dataset, we demonstrated that the proposed ISDL method significantly improves localization performance.

Author Contributions: Conceptualization, H.L. and X.L.; Methodology, H.L. and D.M.; Software, H.L. and D.M.; Validation, H.L. and D.M.; Investigation, H.L.; Writing—original draft, H.L. and D.M.; Writing—review and editing, H.L., D.M. and X.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The *UJIndoorLoc* dataset is available at https://geotec.uji.es/2014/10/03/ujiindoorloc-database/, accessed on 2 January 2025.

Conflicts of Interest: Author Dongfang Mao was employed by the company Wuxi Realid Technology Co., Ltd. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Liu, F.; Liu, J.; Yin, Y.; Wang, W.; Hu, D.; Chen, P.; Niu, Q. Survey on WiFi-based indoor positioning techniques. *IET Commun.* **2020**, *14*, 1372–1383. [CrossRef]
2. He, S.; Chan, S.H.G. Wi-Fi fingerprint-based indoor positioning: Recent advances and comparisons. *IEEE Commun. Surv. Tutor.* **2015**, *18*, 466–490. [CrossRef]
3. Thomas, F.; Ros, L. Revisiting trilateration for robot localization. *IEEE Trans. Robot.* **2005**, *21*, 93–101. [CrossRef]
4. Boquet, G.; Boquet-Pujadas, A.; Pisa, I.; Dabak, A.; Vilajosana, X.; Martinez, B. Indoor position estimation using angle of arrival measurements: An efficient multi-anchor approach with outlier rejection. *IoT* **2024**, *26*, 101236. [CrossRef]
5. Gan, Y.; Cong, X.; Sun, Y. Refinement of TOA localization with sensor position uncertainty in closed-form. *Sensors* **2020**, *20*, 390. [CrossRef]
6. Zafari, F.; Gkelias, A.; Leung, K.K. A survey of indoor localization systems and technologies. *IEEE Commun. Surv. Tutor.* **2019**, *21*, 2568–2599. [CrossRef]
7. Florio, A.; Avitabile, G.; Coviello, G. Digital phase estimation through an I/Q approach for angle of arrival full-hardware localization. In Proceedings of the 2020 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS), Ha Long, Vietnam, 8–10 December 2020; pp. 106–109.
8. Pasku, V.; De Angelis, A.; De Angelis, G.; Arumugam, D.D.; Dionigi, M.; Carbone, P. Magnetic field-based positioning systems. *IEEE Commun. Surv. Tutor.* **2017**, *19*, 2003–2017. [CrossRef]
9. Yang, S.H.; Jung, E.M.; Han, S.K. Indoor location estimation based on LED visible light communication using multiple optical receivers. *IEEE Commun. Lett.* **2013**, *17*, 1834–1837. [CrossRef]
10. Khan, D.; Cheng, Z.; Uchiyama, H.; Ali, S.; Asshad, M.; Kiyokawa, K. Recent advances in vision-based indoor navigation: A systematic literature review. *Comput. Graph.* **2022**, *104*, 24–45. [CrossRef]
11. Singh, J.; Tyagi, N.; Singh, S.; Ali, F.; Kwak, D. A systematic review of contemporary indoor positioning systems: Taxonomy, techniques, and algorithms. *IEEE IoT J.* **2024**, *11*, 34717–34733. [CrossRef]
12. El-Sheimy, N.; Li, Y. Indoor navigation: State of the art and future trends. *Satell. Navig.* **2021**, *2*, 7. [CrossRef]
13. Kim Geok, T.; Zar Aung, K.; Sandar Aung, M.; Thu Soe, M.; Abdaziz, A.; Pao Liew, C.; Hossain, F.; Tso, C.P.; Yong, W.H. Review of indoor positioning: Radio wave technology. *Appl. Sci.* **2020**, *11*, 279. [CrossRef]
14. Xiao, J.; Zhou, Z.; Yi, Y.; Ni, L.M. A survey on wireless indoor localization from the device perspective. *ACM Comput. Surv.* **2016**, *49*, 1–31. [CrossRef]
15. Scavino, E.; Abd Rahman, M.A.; Farid, Z.; Ahmad, S.; Asim, M. Enhancing Indoor Positioning Accuracy with WLAN and WSN: A QPSO Hybrid Algorithm with Surface Tessellation. *Algorithms* **2024**, *17*, 326. [CrossRef]
16. Zanella, A. Best practice in RSS measurements and ranging. *IEEE Commun. Surv. Tutor.* **2016**, *18*, 2662–2686. [CrossRef]
17. Prasad, K.S.V.; Hossain, E.; Bhargava, V.K. Machine learning methods for RSS-based user positioning in distributed massive MIMO. *IEEE Trans. Wireless Commun.* **2018**, *17*, 8402–8417. [CrossRef]
18. Cheng, L.; Zhao, P.; Wei, D.; Wang, Y. A robust indoor localization algorithm based on polynomial fitting and Gaussian mixed model. *China Commun.* **2023**, *20*, 179–197. [CrossRef]
19. Guivant, J.; Narula, K.; Kim, J.; Li, X.; Khan, S. Compressed Gaussian Estimation under Low Precision Numerical Representation. *Sensors* **2023**, *23*, 6406. [CrossRef]
20. Tseng, C.H.; Tsaur, W.-J. FFK: Fourier-Transform Fuzzy-c-means Kalman-Filter Based RSSI Filtering Mechanism for Indoor Positioning. *Sensors* **2023**, *23*, 8274. [CrossRef]
21. Umair, S.M.A.; Arslan, T. A Narrow-Down Approach Based on Machine Learning for Indoor Localization. *Algorithms* **2023**, *16*, 529. [CrossRef]
22. Torres-Sospedra, J.; Richter, P.; Moreira, A.; Mendoza-Silva, G.M.; Lohan, E.S.; Trilles, S. A comprehensive and reproducible comparison of clustering and optimization rules in wi-fi fingerprinting. *IEEE Trans. Mobile Comput.* **2020**, *21*, 769–782. [CrossRef]
23. Cha, J.; Lim, E. A hierarchical auxiliary deep neural network architecture for large-scale indoor localization based on Wi-Fi fingerprinting. *Appl. Soft Comput.* **2022**, *120*, 108624. [CrossRef]

24. Ibrahim, M.; Torki, M.; ElNainay, M. CNN based indoor localization using RSS time-series. In Proceedings of the In 2018 IEEE Symposium on Computers and Communications (ISCC), Natal, Brazil, 25–28 June 2018; pp. 01044–01049.
25. Ahmed Elesawi, A.E.; Kim, K.S. Hierarchical Multi-Building And Multi-Floor Indoor Localization Based On Recurrent Neural Networks. In Proceedings of the 2021 Ninth International Symposium on Computing and Networking Workshops (CANDARW), Matsue, Japan, 23–26 November 2021; pp. 193–196.
26. Naoumi, S.; Bazzi, A.; Bomfin, R.; Chafii, M. Complex Neural Network based Joint AoA and AoD Estimation for Bistatic ISAC. *IEEE J. Sel. Top. Signal Process* **2024**, 1–15. [CrossRef]
27. Njima, W.; Chafii, M.; Shubair, R.M. GAN Based Data Augmentation for Indoor Localization Using Labeled and Unlabeled Data. In Proceedings of the 2021 International Balkan Conference on Communications and Networking (BalkanCom), Novi Sad, Serbia, 20–22 September 2021; pp. 36–39.
28. Yoo, J. Wi-Fi Fingerprint Indoor Localization by Semi-Supervised Generative Adversarial Network. *Sensors* **2024**, *24*, 5698. [CrossRef] [PubMed]
29. Agah, N.; Evans, B.; Meng, X.; Xu, H. A Local Machine Learning Approach for Fingerprint-based Indoor Localization. In Proceedings of the SoutheastCon 2023, Orlando, FL, USA, 1–16 April 2023; pp. 240–245.
30. Li, D.; Zhang, B.; Li, C. A feature-scaling-based k-nearest neighbor algorithm for indoor positioning systems. *IEEE Internet Things J.* **2016**, *3*, 590–597. [CrossRef]
31. Xia, S.; Liu, Y.; Yuan, G.; Zhu, M.; Wang, Z. Indoor fingerprint positioning based on Wi-Fi: An overview. *ISPRS Int. J. Geo-Inf.* **2017**, *6*, 135. [CrossRef]
32. Shin, B.; Lee, J.H.; Lee, T.; Kim, H.S. Enhanced weighted K-nearest neighbor algorithm for indoor Wi-Fi positioning systems. In Proceedings of the 2012 8th International Conference on Computing Technology and Information Management (NCM and ICNIT), Seoul, Republic of Korea, 24–26 April 2012; pp. 574–577.
33. Li, B.; Salter, J.; Dempster, A.G.; Rizos, C. Indoor positioning techniques based on wireless LAN. In Proceedings of the 1st IEEE International Conference Wireless Broadband & Ultra Wideband Communications, Sydney, NSW, Australia, 13–15 March 2006; pp. 3–16.
34. Lee, J.H.; Delbruck, T.; Pfeiffer, M. Training deep spiking neural networks using backpropagation. *Front. Neurosci.* **2016**, *10*, 508. [CrossRef] [PubMed]
35. Nabati, M.; Ghorashi, S.A. A real-time fingerprint-based indoor positioning using deep learning and preceding states. *Expert Syst. Appl.* **2023**, *213*, 118889. [CrossRef]
36. Torres-Sospedra, J.; Montoliu, R.; Martínez-Usó, A.; Avariento, J.P.; Arnau, T.J.; Benedito-Bordonau, M.; Huerta, J. UJIIndoorLoc: A new multi-building and multi-floor database for WLAN fingerprint-based indoor localization problems. In Proceedings of the 2014 International Conference on Indoor Positioning and Indoor Navigation (IPIN), Busan, Republic of Korea, 27–30 October 2014; pp. 261–267.
37. Moreira, A.; Nicolau, M.J.; Meneses, F.; Costa, A. Wi-Fi fingerprinting in the real world-RTLS@ UM at the EvAAL competition. In Proceedings of the 2015 International Conference on Indoor Positioning and Indoor Navigation (IPIN), Banff, AB, Canada, 13–16 October 2015; pp. 1–10.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.